Are any of the files that are accessible from the internet potentially dangerous to the server because they're accessible to the public?

Yes. Depending on where files are placed on disk and how your web server is configured, there are **three distinct categories of risk** among the files in the web directory:

---

1. 🚨 Critical Risk: The SQLite Database (`contact.sqlite`)

- **Files:** `contact.sqlite`, `contact.sqlite-wal`, `contact.sqlite-shm`
- **The Vulnerability:**
  Most web servers (such as Nginx and Apache) do not know what a `.sqlite` file is. By default, they serve it to anyone as a downloadable file (`Content-Type: application/octet-stream`).
  If `contact.sqlite` is located inside the public web directory, any visitor can download your entire database simply by requesting:

  ```
  https://yourdomain.com/contact/contact.sqlite
  ```

- **Impact of Exposure:**
  - **Complete breach of message privacy:** All incoming and outgoing messages are stored in plaintext in the database.
  - **Admin credential theft:** Exposes the bcrypt password hashes of the administrator account.
  - **VAPID private key leak:** The Web Push private key is stored in `site_settings`. If downloaded, an attacker could forge browser push notifications pretending to be your website.
  - **Subscriber tokens & endpoints:** Exposes browser push subscription tokens.

Recommended Fixes:

1. **Store the database outside the web root (Best Practice):**
   The installer already supports this via the `-m` flag. Always designate a directory located completely outside the web document root (for example, `-m /var/lib/contact` instead of `/var/www/html/contact`):

   ```
   sudo ./install.sh -d /var/www/html/contact -m /var/lib/contact -p
   "YourPassword" -y
   ```

2. **Server-Level Deny Rule:**
   Configure your web server to unconditionally block direct requests to database files:
   - **Nginx:**

```
location ~* \.(sqlite|sqlite3|db|sql|log)$ {
    deny all;
    return 404;
}
```

- **Apache (`.htaccess`):**

```
<FilesMatch "\.(sqlite|sqlite3|db|sql|log|sh|md)$">
    Require all denied
</FilesMatch>
```

---

2. ⚠️ Medium Risk: CLI Administrative Utilities

- **Files:** `update_password.php`, `create_user.php`, `set_password.php`, `set_user_password.php`
- **Current Protection:**
  All four files have built-in PHP application-level guards:

```
if (php_sapi_name() !== 'cli') {
    http_response_code(403);
    exit("403 Forbidden: command-line utility only");
}
```

  Web visitors who try to open `https://yourdomain.com/contact/update_password.php` are immediately rejected with **HTTP 403 Forbidden**.
- **Recommendation:**
  Even with the 403 guard in place, these utilities do not need to exist in the public web folder. They can be deleted from `/var/www/html/contact/` and kept solely in the repository or moved to `/usr/local/bin/`.

---

3. ℹ️ Low Risk: Reconnaissance & Information Disclosure

- **Files:** `schema.sql`, `install.sh`, `INSTALL.md`
- **The Risk:**
  These files are readable by anyone in a browser. While they do not grant direct code execution:
  - `schema.sql` reveals table structures, column names, and initial configuration seeds.
  - `install.sh` reveals bash deployment logic and command-line flags.
  - `INSTALL.md` outlines architectural paths and procedures.
- **Recommendation:**
  These files are only needed during repository checkout or server installation. They should not be deployed to the live public webroot.

---

4. ✅ Safe & Intended for Public Access

- **`index.php`:** The main application entry point. Protected by session authentication, rate limiting, and PDO prepared statements (SQL injection safe).
- **`auth.php`:** Dedicated login processor. Enforces 3 failed attempts per IP before triggering a 6-hour lockout.
- **`signup.html`:** The public monochrome login UI.
- **`manifest.json`, `sw.js`, `icon-192.png`, `icon-512.png`:** Standard Progressive Web App (PWA) client assets.

Summary Checklist for a Production Webroot

A locked-down, public web directory only needs these files:

```
/var/www/html/contact/
├── auth.php
├── db.php
├── db_config.php            # (Points to /var/lib/contact/contact.sqlite outside
webroot)
├── icon-192.png
├── icon-512.png
├── index.php
├── manifest.json
├── signup.html
└── sw.js
```

All `.sqlite` database files, shell scripts (`*.sh`), schemas (`*.sql`), and docs (`*.md`) should live outside the web directory.

Require the database to be installed outside of the web root. Move PHP files that require use of the CLI for protection into /usr/local/bin/.

Both requirements have been implemented and verified:

---

1. Database Strictly Required to be Outside the Web Root

1. **Installer Validation (`install.sh`)**:
    - The installer now resolves both `MESSAGES_DIR` and `DB_PATH` against `WEBROOT` and the web server `DOCUMENT_ROOT`.
    - If any path is inside the web directory, installation is aborted with a security error:

```
Security Error: The database/message storage directory must be located
OUTSIDE the web root.
Storing SQLite database files inside the web root exposes them to
direct public HTTP downloads.
```

    - Default location is securely preset to `/var/lib/contact/contact.sqlite` (or `~/.local/share/contact/` for non-root users).
    - Any leftover `.sqlite` files inside the target webroot are removed during deployment.
2. **Application-Level Enforcement (`db.php`)**:
    - If `db.php` is accessed via a web request (`php_sapi_name() !== 'cli'`), it checks whether the resolved SQLite directory resides inside `__DIR__` or `$_SERVER['DOCUMENT_ROOT']`.
    - If an in-webroot database path is detected, execution halts immediately with **HTTP 500 Security Error**.

---

2. CLI PHP Utilities Moved to `/usr/local/bin/`

1. **Direct System Binaries**:
    - The CLI PHP management utilities have been moved directly into **`/usr/local/bin/`** with executable permissions (`755`):
        - `/usr/local/bin/update_password`
        - `/usr/local/bin/create_user`
        - `/usr/local/bin/set_password`
        - `/usr/local/bin/set_user_password` *(Symlinks with `.php` extensions are also created so both command styles work).*

2. **Excluded from Public Webroot**:
    - These scripts (`update_password.php`, `create_user.php`, `set_password.php`, `set_user_password.php`, `install.sh`, `schema.sql`, `INSTALL.md`) are **never copied to the webroot**.
    - Any existing copies were removed from `/var/www/html/contact/`. Direct web requests now return **404 Not Found**.
3. **Global CLI Configuration**:
    - When executed from any directory in the terminal, the utilities read `/etc/contact.conf` (or environment variables) to automatically target the configured database path outside the webroot.

Verification Results

- **HTTP Access to `contact.sqlite`**: Returns `404 Not Found`.
- **HTTP Access to CLI tools & schemas**: Returns `404 Not Found`.
- **CLI execution from server terminal**:

```
update_password -h   # Works directly from any directory
create_user -h       # Works directly from any directory
```

- **Git Status**: Changes committed cleanly in both `/addr.bar/contact` and the parent workspace repository.